EduComic

& EduRetrieve

A tool that turns a textbook PDF into comic-strip lessons and quizzes — built on top of a search engine you can actually measure.

Two things in one	A learning app, and the information retrieval engine underneath it
Runs entirely offline	No cloud APIs, no keys — local AI models on your own machine
Built and tested	450 automated tests, 19 page checks
Measured, not claimed	48 test questions, 8 search methods compared with statistics

Prepared 21 October 2026

What is this, in one paragraph?

Reading a textbook is slow, and it is hard to tell whether you have understood anything.

You upload a PDF of a textbook. The system reads it, breaks it into topics, and for each topic it can draw you a short comic strip that explains the idea, plus a quiz to check you got it. You can also just **ask it questions in plain English** and it will find the right passage and write an answer — always showing which part of your book the answer came from.

Underneath, the interesting part is the **search engine**. Finding the right paragraph in a book is a genuinely hard problem, and most projects just assume their search works. This one measures it: there is a test set of questions, a set of standard scoring methods, and statistical tests that say whether one search method is really better than another or just lucky.

> **The short version**
>
> The app is the demonstration. The search engine is the contribution. Everything the comics and quizzes read comes through the same engine that gets measured, so the numbers describe the thing you actually use.

How it works

Two pipelines. The first runs once per book; the second runs when you ask for something.

1. When you upload a book

PDF	You upload a textbook (short ones, 5–10 pages, process fastest)
Read	Text is pulled out page by page
Split	The text is cut into overlapping passages of about 400 words
Index	Each passage is filed two ways: by the words it contains, and by what it means
Topics	A local AI model reads the book and lists the subtopics it covers
Quizzes	Ten multiple-choice questions are written for each subtopic

2. When you ask a question or request a comic

Understand	Work out what kind of question it is, and resolve follow-ups like “why does it increase?”
Search	Two searches run at once — one on words, one on meaning — and the results are merged
Re-rank	A second AI model re-reads the top results next to your question and reorders them
Answer	A local AI model writes an answer using only those passages, with numbered citations
Or draw	For a comic: a storyboard is written, pictures are generated, and captions are drawn on top

What it can do

Grouped by the part of the app you would be using.

Reading and learning

- Upload a textbook PDF and have it split into subtopics automatically
- Generate a comic-strip lesson for any subtopic, in four art styles and three difficulty levels
- Take a ten-question quiz on each subtopic, with explanations for every answer
- Track which subtopics you have studied and how you scored

Asking questions

- Ask anything in plain English — you do not need the textbook's exact wording
- Get a written answer with numbered citations back to the exact passages used
- The system says "this book does not cover that" instead of making something up
- Ask follow-ups like "why does it happen?" and it works out what "it" refers to
- Every result can be expanded to show *why* it was found: which words matched, what each search method scored it, and how far the re-ranker moved it

For the evaluation

- Compare five search methods side by side on the same question
- A built-in results dashboard with scores, timings and significance tests
- A reproducible experiment script that regenerates every number in this document
- Tools to check whether a PDF is usable before you add it

Under the hood

- Runs completely offline on your own machine — no API keys, nothing sent anywhere
- Light and dark themes, works on a phone screen, keyboard accessible
- 450 automated tests, including the search maths checked against hand-calculated values

Why searching a textbook is hard

Five real problems, and what the system does about each. These are not hypothetical — each was observed on the actual textbooks loaded into this project.

The problem	What goes wrong	The fix
You use different words than the book	You ask “why is farmland becoming useless?”; the book says “land degradation”. No word matches.	Match on **meaning** as well as words, and add the book’s own vocabulary to your question
Short terms and abbreviations	Asked “what does CAP mean?”, the meaning-based search returned *capacitor* pages from a physics book.	Keep a **word-matching** search too, and only search one book at a time
Nearly-right answers rank too high	A page on a related topic can outrank the page that actually answers the question.	A second model re-reads the top results **together with** your question and reorders them
A passage makes no sense alone	“This process increases the value by 20%” — which process? Which value?	Show the surrounding text, and say which chapter and subtopic it came from
Follow-up questions have no subject	“Why does it increase?” on its own tells a search engine nothing.	Rewrite it using the conversation: “Why does entropy increase?”

Information retrieval techniques used

The standard IR toolkit, stage by stage. The left column is the name the technique goes by in the literature; the right column is what it actually does here. Everything listed is implemented and running — nothing is aspirational.

Stage 1 — Document processing and indexing

Inverted index	The core data structure of search: a list, for every word, of which passages contain it and how often. Written from scratch rather than taken from a library.
Tokenisation, stopword removal, stemming	Text is lowercased, split into words, common words like “the” are dropped, and endings are trimmed so “capacitors” matches “capacitor”. Both steps can be switched off to measure what they are worth.
Passage chunking with overlap	The book is cut into ~400-word passages that overlap by 80 words, so a sentence split across a boundary still appears whole somewhere.
Document structure / hierarchy	Each passage remembers its page, chapter and subtopic, so a result can say where in the book it came from.

Stage 2 — Retrieval models

TF-IDF vector space model	The classical method. Passages and the query become vectors of word weights, compared by cosine similarity. Used as the baseline everything else is measured against.
BM25 (probabilistic model)	The standard modern word-matching method. Adds two things TF-IDF lacks: repeating a word stops helping after a few times (saturation, the k1 setting), and long passages are not rewarded just for being long (length normalisation, the b setting). Both IDF formulations are implemented.
Dense retrieval (bi-encoder)	Passages and queries are turned into vectors that capture meaning rather than words, so a question can match a passage with no words in common. Two embedding models are supported.

Stage 3 — Rank fusion

Reciprocal Rank Fusion (RRF)	Merges the word-based and meaning-based result lists using only their *positions*, never their scores — which matters because the two score types are on completely different scales and cannot be added directly. Uses the standard k = 60.
CombSUM and CombMNZ score fusion	The alternative approach: normalise the scores, then add them. CombMNZ additionally rewards a passage both methods agree on. Implemented so RRF can be compared against them rather than just asserted to be better.
Score normalisation (min-max, z-score)	Rescales one method's scores so they can sit alongside another's. Needed by score fusion; not needed by RRF.

Stage 4 — Re-ranking

Cross-encoder neural re-ranking	A second model re-reads the top ~20 results *together with* the question, rather than scoring each in isolation. Slower, but the single biggest quality gain in the results table.

Stage 5 — Query processing

RM3 pseudo-relevance feedback	The classical query-expansion method: assume the first few results are relevant, harvest their distinctive words, and search again with those added.
LLM query expansion	The modern alternative: ask a language model what terminology the textbook would use. Unlike RM3 it can help even when the first search found nothing.
Query intent classification	Works out whether you are asking for a definition, a comparison, a cause, a process, a number or an example — seven types, detected by sentence pattern.
Intent-aware adaptive retrieval	Uses that intent to change the search: lean on word-matching for definitions, on meaning for "why" questions.
Multi-query / entity splitting	"Difference between X and Y" is also searched as X and as Y separately, so both sides are guaranteed to appear.
Conversational query rewriting	Turns "why does it increase?" into "why does entropy increase?" using the conversation.

Stage 6 — Evaluation

Test collection construction	A fixed set of questions with marked-correct passages, built automatically and in two phrasings — one using the book's words, one deliberately avoiding them.
Graded relevance judgements	Passages are marked fully relevant, partly relevant, or not relevant, rather than a simple yes/no.
Ranked-retrieval metrics	Precision@k, Recall@k, F1, Success@k, MRR, MAP and nDCG — all implemented from scratch and checked against hand-calculated values.
Significance testing	A paired randomisation test decides whether one method really beat another or was just lucky. Holm correction accounts for running many comparisons at once.
Ablation studies	Each component is removed in turn to measure what it contributes — which is how two features were found to be making things worse.

Stage 7 — Explainability

Per-term score decomposition	For any result, shows each matched word with its frequency, rarity and exact contribution to the score.
Fusion contribution shares	Shows how much of a result's final position came from word-matching versus meaning-matching.
Rank-change tracking	Shows how far the re-ranker moved each result, up or down.

> **Why both old and new methods are here**
>
> Classical and neural methods are both implemented, and compared against each other rather than one being assumed better. TF-IDF and BM25 are the textbook baselines; dense retrieval and the cross-encoder are the modern additions; RRF is what lets them be used together.

Does it actually work?

Eight search methods were run over the same 48 test questions. Higher is better; the score runs from 0 to 1.

Search method	Score	First hit	Speed	Real gain?
Hybrid + re-ranker	**0.702**	0.769	908 ms	**yes**
Full pipeline (everything)	**0.693**	0.768	960 ms	**yes**
Hybrid (score fusion)	**0.647**	0.697	9 ms	**yes**
Hybrid (both, fused)	**0.644**	0.704	10 ms	—
Dense (meaning matching)	**0.639**	0.695	10 ms	—
Hybrid + word expansion	**0.619**	0.658	11 ms	—
BM25 (better word matching)	**0.571**	0.613	0 ms	—
TF-IDF (classic word matching)	**0.568**	0.623	1 ms	—

“Score” is nDCG@10, the standard measure for how well a ranked list of results is ordered. “First hit” is how near the top the first good result appears. “Real gain?” means the improvement over the simplest method survived a statistical test — without it, a higher number could just be luck.

What the table says

Combining word-matching with meaning-matching, then re-reading the top results, scores **0.702** against **0.568** for the classic method — and the gap is statistically real, not noise. The cost is speed: the re-ranker takes about 0.9 seconds per question against roughly 10 milliseconds without it.

What it cannot do yet

Stated plainly, because a project that only lists its strengths is harder to trust.

- **Scanned PDFs do not work.** A scanned book is a stack of photographs with no text inside, so nothing can be searched. The app detects this and says so rather than failing quietly.
- **The test set is small.** 48 questions over a small library. The differences that passed the statistical test are real, but the ones that did not are undecided rather than disproved.
- **Two features did not help.** Automatic word expansion and question-type routing both made results slightly *worse* on this collection. They are kept, switched off, and reported as negative findings.
- **Comics need a graphics card.** Question answering runs on any machine; drawing comic panels needs a local image-generation setup.
- **Answers are only as good as the book.** If the textbook is wrong, so is the answer — which is why every answer cites the passage it used, so you can check.

What it is built from

Web app	Django (Python)
Word search	BM25 and TF-IDF, written from scratch over an inverted index
Meaning search	Sentence embeddings (MiniLM), 384 dimensions
Re-ranking	A cross-encoder model (MS MARCO MiniLM)
Writing and reasoning	A local language model via Ollama
Comic artwork	Stable Diffusion via ComfyUI
Captions and bubbles	Drawn with Pillow, so text is never garbled by the image model

Running it

Start the app	`venv\Scripts\python manage.py runserver`
Background worker (for comics)	`venv\Scripts\python manage.py qcluster`
Re-run the experiment	`venv\Scripts\python tools\run_experiments.py`
Check a PDF is usable	`venv\Scripts\python tools\check_pdfs.py media\textbooks`

Generated from the live project. The results table is read from the most recent saved experiment run, so this document reflects the current state of the code rather than a snapshot typed by hand.